QuantRule - Deployment and Operations

1. The operator interface

```
make up      # build, start, wait for health, print the URL
make down    # stop, keep data
make logs    # follow
make ps      # status
make scale N=4
make verify  # print the canonical digest
make reset   # stop and delete volumes, with confirmation
make remote  # publish only :8080 behind a token
make backup  # dump Postgres and mirror objects
make restore FILE=...
```

`make up` blocks until `/health` returns 200 and prints:

```
QuantRule v1.0.0 ready -> http://localhost:8080
```

2. Topology

Container	Image	Published	Volume
`gateway`	`quantrule-server`	`8080`	-
`data`	`quantrule-server`	-	`config/` read-only
`model`	`quantrule-server`	-	-
`backtest`	`quantrule-server`	-	-
`evidence`	`quantrule-server`	-	-
`simulation`	`quantrule-server`	-	`config/` read-only
`postgres`	`postgres:17`	-	`pgdata`
`minio`	`minio/minio`	-	`objects`
`rabbitmq`	`rabbitmq:4-management`	-	`mqdata`

One bridge network. Services resolve each other by container name. Only the gateway is reachable from the host.

Nine containers, six service binaries, one image digest.

3. Configuration

Environment variables only; no file is required to start.

Variable	Default	Purpose
`QR_IMAGE_DIGEST`	image label	Reported by `/health`, written to provenance
`QR_DATABASE_URL`	compose value	Postgres
`QR_OBJECT_STORE_URL`	compose value	MinIO
`QR_QUEUE_URL`	compose value	RabbitMQ
`QR_UPLOAD_LIMIT_BYTES`	`536870912`	Upload cap
`QR_BEARER_TOKEN`	empty	Enables the token check
`QR_BACKTEST_CONCURRENCY`	`1`	Runs per worker

Repository configs are mounted read-only at `config/` and catalogued beside uploads. Every config receives a `config_hash`.

4. Health and failure

`GET /health` aggregates each service, the queue depth, and the worker count.

Situation	Operator action
Run queued too long	Check worker count and queue depth
Run running too long	Read the stage, cancel if needed
Run failed	Read the failed stage and code, fix the input, resubmit
Dead-lettered	Inspect the attempt count; the run row is the record
Stuck running after a crash	Restart the worker; finalization retries

5. Backup and recovery

`make backup` writes one timestamped archive: a Postgres dump and an object mirror. `make restore` verifies every artifact hash and stops on a mismatch.

Incident	First response
Disk full	Stop workers, free object storage, restart
Database corruption	Restore the last backup; record the data-loss window
Object store loss	Restore from backup; missing artifacts make runs unverifiable
Queue loss	Rebuild from run rows in `queued`
Image regression	Roll back to the previous digest

6. Upgrade and rollback

1. Record the current image digest.

2. `make down`, pull or rebuild, `make up`.
3. Migrations run forward at startup and are idempotent.
4. `make verify` and compare with the recorded digest.
5. Rollback starts the previous digest. Migrations are forward-only, so the previous version must tolerate the newer schema.

7. Remote exposure

A published gateway port needs one gate: a private network (WireGuard, Tailscale) or a bearer token with TLS. `make remote` sets the token requirement and publishes only `:8080`.

8. What operations does not provide

No high availability, no failover, no autoscaling, no multi-host scheduling, no service mesh, no distributed tracing infrastructure. Each is a later decision with its own record.

9. Release

A release is an image digest and a short note. The release note records the digest; the desktop asset, when present, carries the same version. The image is checked for the absence of a Python interpreter before publication.